Deliveroo.js: a BDI and LLM Solution with PDDL Planning for Autonomous Parcel Delivery

Autonomous Software Agents — Final Project Report

A.A. 2025–2026 University of Trento

Filippo Roseghini ID: 267934. roseghini.filippo@studenti.unitn.it

Abstract

This report presents the design and implementation of two autonomous agents for **Deliveroo.js**, a competitive multi-agent parcel delivery game. **Agent A** follows a Belief–Desire–Intention (BDI) architecture with utility-based option generation, A* pathfinding, and a three-level PDDL planning hierarchy covering tile navigation, task sequencing, and Sokoban-style crate recovery. **Agent B** implements a Plan-and-Execute architecture backed by a large language model (Llama-3.3-70B via LiteLLM), enabling free-text mission interpretation with near-zero perceived latency through a hybrid startup mechanism. The two agents coordinate via a lightweight message-passing protocol over the game chat bus, supporting parcel handoff, relay mode, and team-wide constraint synchronisation.

Keywords: BDI, PDDL, LLM, Multi-Agent Systems, Plan-and-Execute, Sokoban Planning, Deliveroo.js

Contents

1 Introduction **1**

2 Agent A: BDI Architecture **2**
2.1 Environment Perception and Belief Revision . . . 2
2.2 Desires: Option Generation and Utility Scoring . . . 2
2.3 Intentions: IntentionRevisionReplace and Plan Library . . . 3
2.4 A* Pathfinder . . . 4

3 Agent B: LLM Agent and Multi-Agent Coordination **4**
3.1 Architecture: Plan-and-Execute . . . 4
3.2 Challenge 2: Atomic Missions (Level 1) . . . 5
3.3 Challenge 2: Persistent Strategy Constraints (Level 2) . . . 5
3.4 Multi-Agent Coordination (Level 3) . . . 5

4 PDDL Planning **6**
4.1 Navigation Planner (`pddlPlanner.js`) — tile-level STRIPS . . . 6
4.2 Task-Level Delivery Planner (`pddlDelivery.js`) — POI graph . . . 6
4.3 Sokoban Crate Planner (`pddlCrates.js`) — push-sequence recovery . . . 6
4.4 Complexity Discussion . . . 7
4.5 Practical Trade-off . . . 7

5 Validation and Testing **8**
5.1 Agent A: BDI on Standard and Crate Maps . . . 8
5.2 Agent B: LLM Agent under Mission Variety . . . 8
5.3 Multi-Agent Coordination . . . 9

6 Conclusions **9**

References **10**

1 Introduction

This report documents the design and implementation of two autonomous software agents for **Deliveroo.js** [6], a web-based multi-agent game in which agents compete to maximise score by collecting and delivering parcels on a 2-D tile grid. Parcels decay in value over time, placing a premium on rapid pick-up/delivery cycles and efficient path-planning. The environment also features directional tiles (conveyor belts), crates (Sokoban-style movable obstacles), multiple delivery and spawn zones, and dynamic competition with other agents.

We developed two agents with complementary architectures:

- **Agent A (BDI):** a classic Belief–Desire–Intention agent with A* pathfinding, real-time utility reasoning, and a PDDL recovery layer for Sokoban-style obstacle navigation (`agente3/`).
- **Agent B (LLM):** a Plan-and-Execute agent backed by a large language model (Llama-3.3-70B via LiteLLM [4]) capable of interpreting free-text mission objectives, issuing deterministic navigation actions, and replanning on failure (`agente_llm/`).

Both agents share the same belief representation (`beliefs.js`) and A* planner (`planner.js`) and coordinate through a lightweight message-passing protocol (`[COORD]`) over the game's chat bus.

Figure 2: System overview (render with mermaid.live)

```
flowchart LR
    Server["DeliverooJS Server"]
    subgraph AgentA ["Agent A     BDI (agente3
        /)"]
        direction TB
        BelA["beliefs.js\n(Beliefs)"]
        PlanA["planner.js\n(A*)"]
        AgA["agent.js\n(BDI loop)"]
        PDDL["pddlPlanner / pddlDelivery\n/
            pddlCrates"]
    end
    subgraph AgentB ["Agent B     LLM (
        agente_llm/)"]
        direction TB
        BelB["beliefs.js\n(Beliefs)"]
        PlanB["planner.js\n(A*)"]
        AgB["agent_llm.js\n(Plan-and-Execute)"]
        LLM["LiteLLM\n(Llama-3.3-70B)"]
    end
    Server -- "onSensing / onMap /\nonConfig /
        onYou" --> AgentA
    Server -- "onSensing / onMap /\nonConfig /
        onYou" --> AgentB
    AgentA -- "emitMove / emitPickup\n/
        emitPutdown" --> Server
    AgentB -- "emitMove / emitPickup\n/
        emitPutdown" --> Server
    AgentA <-- "[COORD] messages\n(game chat)"
        --> AgentB
```

2 Agent A: BDI Architecture

Agent A is designed around the classic BDI model [1], in which the agent continuously cycles through belief revision, desire generation, and intention selection.

2.1 Environment Perception and Belief Revision

Agent A maintains a continuously updated world model in `beliefs.js` (`class Beliefs`). Every `socket.onSensing` event triggers `updateSensing(sensings, myPosition, sensingRadius)`, which applies the following revision policy to the agent's belief state:

- **Parcel beliefs.** Parcels seen in the current sensing window are inserted or updated (position, reward, timestamp `lastSeen`). Parcels *inside* the sensing radius but absent from `sensings` are removed (they have been picked up or expired). Parcels *outside* the radius are retained until a TTL derived from the server's decay rate expires, preventing premature deletion of high-value parcels momentarily out of view.
- **Agent beliefs.** Other agents observed in the sensing window are recorded by `id` with their last known position. Stale entries older than 30 s (`AGENT_MEMORY_MS`) are purged.
- **Crate beliefs.** Crate positions are tracked in a `Set` (`Beliefs.crates`). Changes are only processed when a crate tile is inside the sensing radius, preserving the last known position of out-of-range crates as obstacles.
- **Map belief.** The static map is populated once from `socket.onMap` and never overwritten. Delivery zones and spawn zones are extracted at load time into dedicated arrays.
- **Sensing radius calibration.** `sensingRadius` is initialised from `process.env.SENSING_RADIUS` (fallback: 5) and updated dynamically via `socket.onConfig`, reading `config.GAME.player.observation_distance`. This ensures correct belief revision boundaries regardless of which server map is loaded.

2.2 Desires: Option Generation and Utility Scoring

`optionsGeneration()` is invoked on every sensing cycle and produces exactly one *predicate* (the intended next action). Candidate actions are: `go_pick_up`, `go_deliver`, `patrol`, `crate_nav`, and coordination predicates (`coord_idle`, `coord_goto_wait`, `coord_rendezvous`).

Pickup utility

For each visible, non-blacklisted parcel p the agent computes:

$$U_{\text{pickup}}(p) = \frac{\hat{R}(p, d_p)^2}{d_p}$$

where d_p is the Manhattan distance to p and $\hat{R}(p, d_p)$ is the estimated reward at arrival, accounting for the decay already accumulated since the last sensing and the decay that will accrue during the d_p-step walk:

$$\hat{R}(p, d) = \max\!\big(0,\ r_p - (\Delta t_{\text{elapsed}} + d \cdot T_{\text{step}}) \cdot \lambda\big)$$

with λ the per-millisecond decay rate (from `Beliefs.getDecayRatePerMs`) and T_{step} the per-move duration (auto-calibrated via EMA from actual `emitMove` timings). Parcels whose estimated arrival reward is ≤ 0 are discarded entirely.

Delivery utility

When the agent carries at least one parcel it evaluates each valid delivery zone z for:

$$U_{\text{deliver}}(z) = \frac{R_{\text{carried}}(t)}{d_z}$$

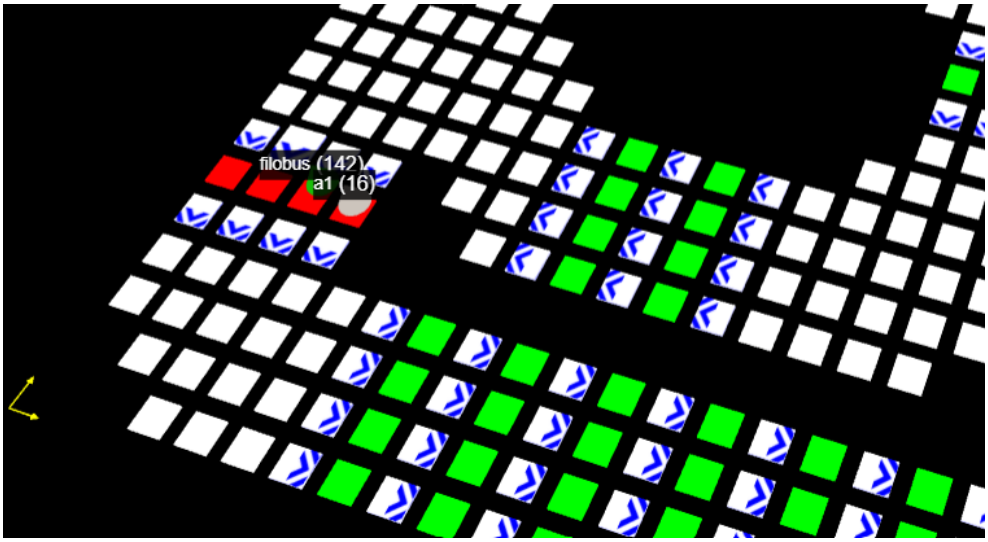


Figure 1: Agent A (**filobus**, red, BDI, score 142) competing against a random agent (**a1**, white, score 16) on a map with directional conveyor tiles (blue arrows) and multiple delivery zones (green). The score gap illustrates the advantage of utility-driven option generation over random behaviour in the same environment.

where $R_{\text{carried}}(t)$ is the sum of currently carried rewards adjusted for elapsed decay, and d_z is the Manhattan distance to zone z.

Decision policy

The agent delivers if (a) it is at capacity (`MAX_CAPACITY` = 18 parcels), (b) the batch size constraint from a team mission is satisfied, or (c) $U_{\text{deliver}} > U_{\text{pickup}}$ and it already holds parcels. Patrol is chosen as a fallback when no parcel is visible and the agent has not recently visited a spawn zone.

Blacklist and congestion map

Unreachable parcels and delivery zones are temporarily banned (`BAN_DURATION_MS` = 3 s, `blacklist.parcels / .zones`) to avoid oscillation. Tile-level move rejections are recorded in a `congestionMap` (TTL = 5 s, penalty = 100) that is passed to the A* planner as an additive cost term, steering the agent around dynamically occupied tiles without permanent map changes.

2.3 Intentions: IntentionRevisionReplace and Plan Library

We implement the **IntentionRevisionReplace** pattern: on each `myAgent.push(predicate)` call, if the new predicate is *identical* to the current one, execution continues uninterrupted (intention stability); if it differs, the running plan is stopped and the new predicate is pursued immediately (reactivity). This single invariant avoids a dedicated watchdog timer and makes the agent both stable under sustained pursuit of the same objective and immediately responsive to better opportunities.

Figure 3: Core of the `IntentionRevisionReplace` class (`agent.js`)

```
async push(predicate) {
    // Stability: identical predicate -> do not
        interrupt the running plan
    if (JSON.stringify(predicate) ===
        JSON.stringify(this.currentPredicate))
            return;

    // Reactivity: new predicate -> stop
        current plan and start the new one
    if (this.currentPlan) this.currentPlan.stop
        ();
    this.currentPredicate = predicate;

    // Scan the plan library for the first
        applicable plan
    for (const PlanClass of planLibrary) {
        if (PlanClass.isApplicableTo(predicate
            [0])) {
            this.currentPlan = new PlanClass();
            await this.currentPlan.execute(...
                predicate);
            return;
        }
    }
    throw new Error(`No plan for predicate: ${
        predicate[0]}`);
}
```

The **Plan Library** contains the following plans, tried in order until one succeeds:

Plan	Trigger predicate	Behaviour
`GoPickUp`	`go_pick_up x y id`	A* to (x, y), `emitPickup`
`GoDeliver`	`go_deliver x y`	A* to (x, y), `emitPutdown`
`Patrol`	`patrol x y`	A* to nearest unvisited spawn zone
`CrateNavigate`	`crate_nav x y`	PDDL Sokoban solver (§4)
`CoordIdle`	`coord_idle`	busy-wait (freeze command active)
`CoordGotoWait`	`coord_goto_wait x y d`	A* to rendezvous tile, wait for release
`CoordRendezvous`	`coord_rendezvous x y`	A* to mid-point for handoff

Plans that fail (A* returns `null`, move rejected `MAX_MOVE_FAILS` times consecutively) propagate the failure to `IntentionRevisionReplace`, which allows the next `optionsGeneration` cycle to choose a different goal.

Figure 4: BDI control loop of Agent A

```
flowchart TD
    S([socket.onSensing]) --> BR[Belief
        Revision\nupdateSensing]
    BR --> OG[optionsGeneration\ncompute
        U_pickup and U_deliver]
    OG --> |"coord mode active"| CM[
        Coordination predicate\ncoord_idle /
        goto_wait / rendezvous]
    OG --> |"hasParcels and U_deliver wins"| GD
        [go_deliver x y]
    OG --> |"U_pickup wins"| chG{
        reachableByAStar?}
    chG --> |yes| GP[go_pick_up x y id]
    chG --> |"no and mapHasCrates"| CN[
        crate_nav x y]
    OG --> |"no parcels visible"| PT[patrol x y
        ]
    GP & GD & PT & CN & CM --> IR[
        IntentionRevisionReplace\nstop if
        predicate changed]
    IR --> PL[Plan Library lookup]
    PL --> EX[Plan.execute\nA* path or PDDL
        sequence]
    EX --> |"emitMove / pickup / putdown"| S
    EX --> |"plan failed"| BL[blacklist target\
        nback to OG]
```

2.4 A* Pathfinder

The A* implementation in `planner.js` models the grid as a 4-connected graph. The heuristic is Manhattan distance (h-consistent, $h(n) = |x_n - x_g| + |y_n - y_g|$). Edge traversal respects two constraints added over the baseline:

1. **Directional tiles.** The tile type encodes the allowed direction (e.g. `'→'` blocks entry from the right). The planner checks `tile.type` directly, fixing a bug in the original scaffold where the non-existent property `tile.direction` made the check always false.
2. **Crates as absolute walls.** Crate positions are passed as a `Set<string>` (`crateKeys`) and treated as impassable, because Agent A's A* does not model push sequences. Multi-push traversal is delegated to the PDDL Sokoban planner (§4).

Before accepting any push that would move a crate, the planner verifies **connectivity invariants** via a multi-source BFS from delivery zones: all spawn zones and the agent's current position must remain reachable after the push (`criticalZones` parameter). This prevents the agent from irreversibly walling itself off from spawn areas.

3 Agent B: LLM Agent and Multi-Agent Coordination

3.1 Architecture: Plan-and-Execute

Agent B follows the **Plan-and-Execute** schema [3]: the LLM is invoked *only* to produce a high-level plan (sequence of tool calls), while all low-level execution (navigation, pickup, delivery) is handled by deterministic code with zero additional LLM calls.

Hybrid startup

Because LLM inference takes 3–8 s, the agent *does not block* at startup. It immediately executes a deterministic fallback plan (top-k parcels by utility, greedy delivery) while `llmPlan()` runs in the background. As soon as the LLM response arrives, the agent adopts it, discarding the fallback mid-flight if necessary. This yields perceived zero latency for the observer while satisfying the requirement that the LLM always drives planning.

LLM-Memory

Before each plan request, `buildLLMMemory()` assembles a structured context block:

- **Objective:** the current mission text (natural language or autonomous).
- **Environment observation** (`get_visible_environment()`): agent position, score, parcels sorted by utility with estimated arrival reward, delivery zone distances, active mission constraints.
- **Agent memory:** last 8 action outcomes (objective → result), kept in a rolling buffer (`agentMemory`). Enables the LLM to avoid repeating recently failed actions.
- **BDI partner status:** last coordination message from Agent A.

Tool catalogue

The LLM receives the following deterministic tools: `navigate_to`, `pick_up`, `put_down`, `drop_here`, `calculate`, `reply_to_sender`, `send_message_to_partner`, `avoid_tiles`, `set_delivery_batch`, `set_delivery_bonus`, `set_zero_value_tiles`, `set_max_parcel_reward`, `clear_mission_rules`, `rendezvous`, `handoff_to_partner`.

Opportunistic pickup

`navigate_to` picks up any parcel on each traversed tile at zero extra LLM cost. This is suppressed when a `deliverBatchSize` constraint is active (to avoid accidentally over-filling the stack) or when a `maxParcelReward` constraint would be violated.

Replanning

If an executed tool returns a failure string (e.g. *"no path to x,y"*, *"parcel expired"*), the agent invokes `llmPlan(memory, replanNote)` with the failure note appended to the context. Replanning is capped at **3 attempts per objective** to bound LLM costs; a fourth failure aborts the objective and logs it to `agentMemory`.

Figure 5: Plan-and-Execute loop of Agent B

```
flowchart TD
    OBJ([New objective\n natural language or
        autonomous]) --> HYB
    HYB[Hybrid startup\nrun deterministic
        fallback\nllmPlan in background] -->
        MEM
    MEM[buildLLMMemory\nobservation + history +
         mission rules] --> LLP
    LLP[llmPlan\n1 LLM call -> PLAN a1..aN] -->
         ADOPT
    ADOPT[Adopt LLM plan\ndiscard fallback if
        running] --> EXEC
    EXEC[Executor\niterate a1..aN, 0 LLM calls]
         --> |"tool result OK"| NEXT
    NEXT{More actions?} --> |yes| EXEC
    NEXT --> |no| DONE([Objective complete\nlog
         to agentMemory])
    EXEC --> |"tool failed"| REPL{replan count\
        n< 3?}
    REPL --> |yes| MEM
    REPL --> |no| DONE
```

3.2 Challenge 2: Atomic Missions (Level 1)

Atomic missions arrive as natural-language text from an external agent via `socket.on('msg')`. Agent B interprets the objective through the LLM and executes the appropriate tool sequence. Notable implementation choices:

- **Arithmetic evaluation.** Coordinates expressed as expressions (e.g. `"move to x=4*2 y=(1+3)*3"`) are evaluated by a safe recursive-descent parser (`safeEvalArithmetic`), which accepts only digits and the four arithmetic operators — no `eval()` is ever called.
- **Directed replies.** `reply_to_sender(msg)` uses `socket.emitSay(activeSenderId, msg)` rather than `emitShout`, routing the answer to the requesting agent with $O(1)$ overhead regardless of spatial proximity.
- **Extreme tile queries.** `getMapExtremes()` precomputes the leftmost, rightmost, topmost, and bottommost walkable tiles once at map load time; the result is injected into the LLM context so the model can resolve missions such as *"drop a package in the leftmost tile"* without guessing grid coordinates.

The following trace illustrates a typical Level 1 interaction. The LLM receives the mission as plain text; its JSON response is parsed by `llmPlan()` and handed directly to the executor without further model calls.

Figure 6: Illustrative LLM interaction for a Level 1 atomic mission

```
=== MISSION ===
"Drop a package in the leftmost tile."

=== LLM RESPONSE (parsed JSON) ===
{
  "thought": "I need to find the leftmost
      walkable tile. From the map
              extremes injected in context:
                  leftmost = (-4, 2).
              I will navigate there and put
                  down one parcel.",
  "plan": [
    { "tool": "navigate_to", "x": -4, "y": 2 },
    { "tool": "put_down" }
  ]
}

=== EXECUTOR TRACE ===
[LLM-EXEC] navigate_to -4 2  -> OK (arrived,
    opportunistic pickup: 0)
[LLM-EXEC] put_down          -> OK (1 parcel
    delivered, score +18)
[LLM-EXEC] plan complete, logging to
    agentMemory
```

3.3 Challenge 2: Persistent Strategy Constraints (Level 2)

Level 2 missions impose *persistent* behavioural constraints: forbidden traversal tiles, exact delivery batch size, preferred or zero-value delivery zones, and maximum parcel reward. These are stored in `missionConstraints` and *enforced mechanically* in the execution layer, not merely described in the LLM prompt:

- `avoid_tiles` injects a penalty of 100 000 into the A* congestion map for the forbidden tile, effectively routing around it without making it completely impassable.
- `set_delivery_batch(N)` blocks `put_down` and disables opportunistic pickup until exactly N parcels are carried.
- `set_delivery_bonus` / `set_zero_value_tiles` bias `bestDeliveryZone()` to prefer or exclude specific tiles.

On every constraint change, `broadcastTeamRules()` serialises a complete `[COORD] rules` snapshot to Agent A, keeping both agents' constraints synchronised without differential updates.

3.4 Multi-Agent Coordination (Level 3)

[COORD] protocol

Agents communicate over the game chat bus using a fixed-prefix protocol (`[COORD]`) to avoid interference with mission messages. Agent B (LLM) acts as the **orchestrator**: it reads mission objectives in natural language and issues directives to Agent A (BDI), which never parses free text. The full command set is:

Command	Semantics
`[COORD] pos x y`	position broadcast (both agents, periodic)
`[COORD] goto_wait x y d`	BDI: move to nearest odd-row tile and wait
`[COORD] freeze`	BDI: halt until released
`[COORD] release`	both agents: resume normal play
`[COORD] rules k=v ...`	team-wide constraint snapshot (idempotent)
`[COORD] fetch id x y`	BDI: pick up parcel id at (x, y) (handoff)
`[COORD] arrived`	sender has reached the rendezvous point

Closest-commits parcel allocation

Both agents independently apply the **closest-commits** heuristic (slide 9): parcel p is allocated to the agent with the lower Manhattan distance to it; ties are broken by agent ID (lower ID wins). No central allocator is needed; each agent locally decides whether to pursue or ignore p.

Handoff and relay mode

For missions requiring inter-agent parcel transfer, Agent B executes `handoff_to_partner()`: it drops carried parcels at a midpoint tile, sends a `[COORD] fetch id x y` for each dropped parcel, and activates `relayMode`. In relay mode, Agent B collects parcels but never delivers them, instead continually dropping them for Agent A; Agent A ignores all non-fetch objectives and focuses solely on pickup and delivery.

Figure 7: Coordination sequence for a handoff mission

```
sequenceDiagram
    participant LLM as Agent B (LLM)
    participant BDI as Agent A (BDI)
    participant SRV as DeliverooJS Server

    Note over LLM: receives NL mission via
        socket.msg
    LLM->>LLM: llmPlan -> [handoff_to_partner]
    LLM->>BDI: [COORD] rules relay=1
    Note over BDI: enters Clean Relay Mode
    LLM->>SRV: emitMove (navigate to midpoint)
    LLM->>SRV: emitPutdown (drop parcels)
    LLM->>BDI: [COORD] fetch p1 x y
    LLM->>BDI: [COORD] fetch p2 x y
    BDI->>SRV: emitMove (navigate to parcel)
    BDI->>SRV: emitPickup
    BDI->>SRV: emitMove (navigate to delivery
        zone)
    BDI->>SRV: emitPutdown
    loop periodic
        BDI->>LLM: [COORD] pos bx by
        LLM->>BDI: [COORD] pos lx ly
    end
    Note over LLM: next cycle: pick more
        parcels for relay
```

4 PDDL Planning

We integrate PDDL [2] planning at **three distinct levels** of abstraction, each addressing a different planning sub-problem. All three modules call the `@unitn-asa/pddl-client` library (`onlineSolver`) to submit a domain+problem pair to the `solver.planning.domains` service [5].

4.1 Navigation Planner (`pddlPlanner.js`) — tile-level STRIPS

When the environment variable `USE_PDDL_PLANNER=true`, Agent A replaces its A* planner with an external PDDL solver. The domain models each walkable tile as an object and encodes movement constraints (including directional arrows) as binary predicates `(right/left/up/down ?from ?to)`. This is a pure STRIPS problem with $O(|T|^2)$ facts in the worst case, kept tractable by a bounding-box scoping strategy (at most `MAX_PDDL_TILES` = 600 tiles).

If the solver fails or times out, execution transparently falls back to the A* planner (no interruption of gameplay). This planner is primarily used to demonstrate PDDL integration; the default runtime uses A* for performance.

4.2 Task-Level Delivery Planner (`pddlDelivery.js`) — POI graph

Activated by `USE_PDDL_DELIVERY=true`, this planner operates at the **means-end reasoning** level of the BDI architecture. Instead of tiling the whole grid, it abstracts the world as a *graph of Points of Interest*: the agent's current position, up to `maxParcels` = 8 high-utility parcel locations, and up to `maxZones` = 4 nearest delivery zones. Real-world travel costs between POIs are pre-computed by the A* planner.

Domain

- **Predicates:** `(at ?loc)`, `(parcel-at ?p ?loc)`, `(carrying ?p)`, `(delivered ?p)`.
- **Actions:** `goto ?from ?to` (moves the agent between reachable POI pairs), `pick-up ?p ?loc` (precondition: at ?loc, parcel-at ?p ?loc), `put-down ?p ?dz` (precondition: carrying ?p, at ?dz, delivery zone).

Problem

The problem encodes the agent's current position as the initial `(at agent-pos)`, lists all parcel facts, and sets the goal to $\bigwedge_{p \in P}$ `(delivered` p`)`. The PDDL solver returns an optimal task-level sequence (optimal w.r.t. number of actions, not travel distance — a known limitation of STRIPS without `:action-costs`). A 30 s cooldown (`PDDL_FAIL_COOLDOWN_MS`) prevents hammering the solver after a network failure.

4.3 Sokoban Crate Planner (`pddlCrates.js`) — push-sequence recovery

This is the highest-complexity PDDL component. It is invoked by the `CrateNavigate` plan when A* returns `null` on a map that contains crates (`mapHasCrates = true`). It computes the full multi-push sequence needed to clear a path to the goal tile.

Activation guard

The planner is never invoked on crate-free maps. On such maps, an A* failure means a genuine wall; calling the Sokoban solver would be both incorrect and wasteful. The `mapHasCrates` flag is set once in `socket.onMap` after

`Beliefs.updateMap` reports `crates.size > 0`.

Domain

- **Predicates:** `(at ?t)`, `(right/left/up/down ?a ?b)` (filtered for directional tiles — used for agent movement), `(adj-right/left/up/down ?a ?b)` (raw adjacency — used for crate push destination, as the server does not apply conveyor constraints to the *destination* of a pushed crate), `(crate-at ?c ?t)`, `(yellow ?t)` (tile that can receive a pushed crate), `(crate-free ?t)` (asserted for every walkable tile not occupied by a crate; mandatory under the STRIPS closed-world assumption).
- **Actions:** `move-right/left/up/down` (precondition: `crate-free` on target tile) and `push-right/left/up/down` (precondition: crate on target tile, yellow and free destination).

The key design decision in this domain is the dual-adjacency scheme: agent movement uses `right/left/up/down` (conveyor-aware filtered predicates), while crate destinations use `adj-right/left/up/down` (raw adjacency), because the server does not apply conveyor constraints to the landing tile of a pushed crate.

Figure 8: PDDL `push-right` action from `pddlCrates.js`

```
; Agent at ?a pushes the crate at ?t1 one step
    right into ?t2.
; Requires: agent can move right to the crate
    (?a -right-> ?t1),
;           and the crate's landing tile (?t2)
    is yellow and free.
(:action push-right
 :parameters (?a ?t1 ?t2)
 :precondition (and (at ?a)
                    (right ?a ?t1)        ;
                        conveyor-aware: agent
                        can step right
                    (crate-at ?t1)        ; a
                        crate occupies the
                        target tile
                    (adj-right ?t1 ?t2)   ; raw
                        adjacency for the
                        landing tile
                    (yellow ?t2)          ; only
                         yellow tiles accept
                        crates
                    (crate-free ?t2))     ;
                        landing tile must be
                        unoccupied
 :effect (and (not (at ?a))   (at ?t1)
              (not (crate-at ?t1)) (crate-at ?
                  t2)
              (crate-free ?t1)    (not (crate-
                  free ?t2))))
```

Scoping

The full map is included if it has ≤ 150 walkable tiles (`MAX_PDDL_CRATE_TILES`). For larger maps a bounding box between start and goal is expanded by a margin of 4–8 tiles in each direction (proportional to the start-to-goal distance), ensuring that crates just outside the naive box are not treated as invisible walls. If the scoped problem exceeds the tile or crate budget, the planner returns `null` and the agent falls back to blacklisting.

Safety validation (`isPlanSafe`)

After the solver returns a plan, we validate it with a *no-regression* connectivity check: the plan is accepted only if the final crate configuration leaves at least as many spawn zones connected to delivery zones (via crate-free paths) as the initial configuration. Formally, let S_0 be the set of spawn zones connected to any delivery zone before the plan executes, and S_f the same set after simulating all pushes. The plan is accepted if and only if $|S_f| \geq |S_0|$.

This formulation correctly handles plans that *temporarily* block connectivity (e.g. moving a crate out of the way and restoring it) because the final configuration is identical to the initial one, giving $|S_f| = |S_0|$. An absolute check ($|S_f| = |\text{all spawn zones}|$) would incorrectly reject such reversible plans on maps where some spawn zones are not connected to delivery zones in the baseline configuration.

Path cache

To avoid redundant solver calls on periodic cycles (same start, goal, and crate configuration), we maintain a static `Map` cache in `PddlCratePlanner` keyed on `startX,startY → goalX,goalY : crateHash`, where `crateHash` is the lexicographically sorted join of all crate positions. Cache invalidation is implicit: if any crate moves, the hash changes and the key misses. Eviction uses FIFO at 30 entries. Only plans that pass `isPlanSafe` are stored.

4.4 Complexity Discussion

- **Navigation planner.** With T tiles the state space is $O(T)$ (agent position), the branching factor is 4, and the problem is solvable in $O(T \log T)$ by A*; the PDDL formulation adds constant overhead from the solver.
- **Delivery planner.** With k parcel POIs and z delivery zones the number of states is $O(2^k \cdot (k + z + 1))$ (carrying bitmask, agent position). In practice $k \leq 8$, giving at most $256 \cdot 13 \approx 3{,}000$ states — well within the solver's capacity.
- **Crate planner.** This is the most complex instance. The state space is $O(|T| \cdot \prod_i |Y_i|)$ where $|Y_i|$ is the number of yellow tiles reachable by crate i. In the worst case (maze with 18 crates, $|Y| \approx 10$ per crate) the theoretical space is enormous; in practice, the online solver finds plans in $\sim$3 s on `crates_maze`. The scoping limit (`MAX_PDDL_CRATES` = 20) caps the branching factor when many crates are present.

4.5 Practical Trade-off

Across the three planners, the only one whose cost-benefit trade-off proved consistently positive was the Sokoban crate planner. On crate-free maps it is never invoked; on Sokoban maps it solves a problem A* simply cannot handle, so the $\sim$3 s solver latency is an unavoidable baseline rather than a regression, and the path cache amortises it to near zero after the first cycle.

The navigation planner (§4.1) and the task-level delivery planner (§4.2) degraded performance in most tested scenarios. Both introduce a round-trip to the online solver

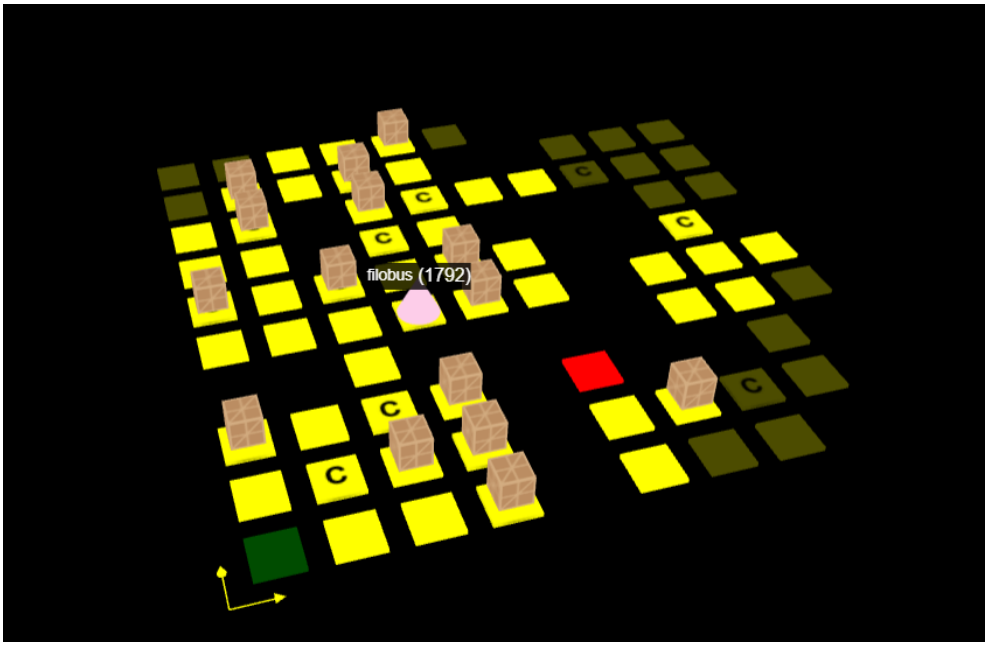


Figure 9: The `crates_maze` map with Agent A (pink, *filobus*) mid-execution of a PDDL Sokoban plan. Brown boxes labelled **C** are crates occupying yellow tiles; the red tile is the only delivery zone; the green tile (bottom-left) is the spawn zone. The agent must push a subset of the 18 crates to open a path to the red delivery zone — a configuration the A* planner cannot solve alone.

— typically 1–5 s — for problems that A* resolves locally in milliseconds. That latency translates directly to missed pickups and avoidable reward decay, making neither planner competitive with the greedy baseline in the environments tested. Both are therefore disabled by default (`USE_PDDL_PLANNER` and `USE_PDDL_DELIVERY` default to `false`) and kept as optional demonstrations of PDDL integration rather than production-grade components.

5 Validation and Testing

Validation was carried out heuristically: both agents were deployed across all available map configurations and exercised through a broad range of scenarios, with the goal of stressing known design boundaries and surfacing failure modes that do not appear in normal play. Agent behaviour was observed directly through console output and the live game view; edge cases were reproduced manually until their root cause was identified.

5.1 Agent A: BDI on Standard and Crate Maps

On maps without crates Agent A behaves robustly. The utility function ($U = \hat{R}^2/d$) consistently favours nearby high-value parcels over distant low-value ones, and the blacklist suppresses oscillation toward temporarily unreachable targets. Patrol fills idle cycles reliably, directing the agent toward spawn zones not yet visited in the current round.

The most relevant failure mode on crate-free maps was the patrol anti-livelock case: on small maps where every spawn zone falls within the sensing radius simultaneously, the `notVisible` guard disqualifies all patrol candidates, leaving the agent stuck in the deadlock-fallback predicate indefinitely. The fix — falling back to the *nearest* spawn regardless of visibility when no `notVisible` candidate exists — was confirmed to resolve the issue on a minimal test map.

`crates_one_way` presented fewer problems than `crates_maze`: the agent navigated and delivered parcels with noticeably less interference from crate-related path failures.

`crates_maze` was the most demanding test, with the PDDL Sokoban solver required frequently to unblock paths. A structural weakness on this map is the interaction between the online solver and the blacklist: if the solver fails or times out, `CrateNavigate` returns `null`, the delivery zone is blacklisted for `BAN_DURATION_MS`, and on `crates_maze` — where the delivery zone is often the only reachable target after a push — the agent idles for the entire ban window with no productive alternative.

5.2 Agent B: LLM Agent under Mission Variety

Agent B was tested with a wide range of natural-language missions to probe both the LLM's plan generation and the executor's mechanical enforcement layer.

Arithmetic coordinate missions (e.g. *"move to x = 4 * 2, y = (1 + 3) * 3"*) were handled correctly in all trials. The LLM consistently identified the expression syntax and passed the raw string to `navigate_to`, which resolves it through `safeEvalArithmetic` rather than evaluating it inline.

Extreme-tile missions (e.g. *"drop a package in the leftmost tile"*) succeeded reliably once the pre-computed

map extremes were injected into the context. The replanning loop served as a useful correction mechanism for LLM inference errors (e.g. targeting a non-walkable tile), as long as the failure is surfaced explicitly by the executor via a descriptive error string.

Batch delivery missions (e.g. *"deliver parcels in stacks of exactly 3"*) were enforced correctly: the `put_down` gate blocked delivery until the carried count reached the batch threshold, and opportunistic pickup was suppressed during transit to prevent overshooting the target. One recurring issue emerged when parcel supply was low: waiting for enough spawns to fill the batch leaves the agent idle, which the LLM sometimes misread as a plan failure, triggering an unnecessary replan that reset the carry count to zero.

Forbidden-tile missions worked reliably. The A* congestion-map penalty of 100 000 routes the agent around the designated tile in all tested configurations, including cases where the tile lies on the natural shortest path. The only notable behaviour was a significantly longer detour when the forbidden tile was the sole stepping-stone between two map areas, which introduced extra decay that the LLM had not budgeted for in its plan.

The most consistent weakness of Agent B is **plan incompleteness**: the model occasionally returned a valid JSON plan that omitted a necessary pickup or intermediate navigation step, requiring a replan. This was most frequent on missions involving multiple parcels across different map areas, where the LLM's greedy nearest-first tendency conflicts with missions that require a specific delivery order or location.

A second weakness is **context drift** on long sessions: the 8-entry rolling memory eventually stops containing early outcomes, and the LLM can revisit strategies it already found ineffective. This manifested during extended relay tests, where the agent occasionally reverted to direct delivery instead of dropping parcels for the BDI partner before the relay constraint in the current prompt corrected its course.

5.3 Multi-Agent Coordination

The closest-commits heuristic was effective whenever the two agents were spatially separated: each targeted a different parcel with no message overhead, and combined throughput was visibly higher than either agent alone. Occasional collisions (both agents committing to the same parcel) were self-correcting, since the slower agent simply moves on to the next candidate in `optionsGeneration`.

The freeze/release primitive was tested with a mission of the form *"stop Agent A and deliver all carried parcels yourself"*: Agent A halted within one sensing tick and Agent B completed the delivery without closest-commits competition.

Relay mode functioned correctly when both agents were active at the same time. When Agent A joined mid-session it missed the initial `[COORD] rules relay=1` broadcast, but the periodic re-broadcast (every 2 s) caused it to enter relay mode within two ticks, an acceptable delay in practice.

The main fragility in the coordination layer is **handoff timing**: if Agent A is mid-movement when a `[COORD] fetch` message arrives, it may complete the current move before processing the command, picking up the wrong parcel first. In practice this causes a one-cycle delay before the correct parcel is retrieved, but on maps with fast reward decay it introduces a measurable score penalty.

6 Conclusions

We have designed and implemented a complete multi-agent system for Deliveroo.js that integrates three complementary planning paradigms: reactive A* pathfinding, deliberative BDI reasoning with utility-based option generation, and symbolic PDDL planning at task, navigation, and Sokoban obstacle levels.

Key results

Agent A demonstrates that a well-tuned utility function (squared reward over distance) combined with intention stability and a blacklist mechanism provides robust, high-throughput behaviour in the default single-agent setting. The graduated PDDL integration (navigation → delivery sequencing → crate recovery) shows that external planners can be introduced at increasing levels of complexity without disrupting the reactive baseline: each PDDL module is gated by runtime flags and fails gracefully. Agent B demonstrates that a Plan-and-Execute LLM agent with hybrid startup achieves near-zero perceived latency while still leveraging the model's natural-language reasoning for free-text mission interpretation, with significantly fewer LLM calls per objective than a per-step reasoning loop would require.

Limitations and future work

- **PDDL optimality.** The delivery planner uses STRIPS without `:action-costs`; the returned plan minimises the number of actions, not travel distance. Switching to `:numeric-fluents` + `:metric minimize (total-cost)` with A*-computed edge costs would yield distance-optimal multi-parcel tours.
- **Path cache scope.** The Sokoban path cache keys on the agent's start position; cycles where the agent approaches the goal from different tiles cause cache misses. Keying only on (goal, crate configuration) and storing the cached plan's start as metadata would allow the executor to enter the cached path mid-sequence, improving hit rates significantly.
- **PDDL solver availability.** The online solver at `solver.planning.domains` introduces a network dependency. A local solver (e.g. Fast Downward) bundled with the agent would eliminate latency variability and offline failures.
- **LLM context drift.** On very long sessions the rolling memory buffer (8 entries) may not suffice to prevent the LLM from repeating errors seen early in the session. A structured episodic memory with semantic retrieval (e.g. a small vector index) would address this.
- **Coordination robustness.** The `[COORD]` protocol is reliable only when both agents are connected simultaneously; a late-joining partner does not automati-

cally receive the current constraint snapshot. Adding a periodic heartbeat re-broadcast of the full `rules` snapshot (already partially implemented) would solve this.

References

References

[1] Rao, A.S. & Georgeff, M.P. (1995). "BDI agents: From theory to practice." *Proc. First International Conference on Multi-Agent Systems (ICMAS)*, 312–319.

[2] McDermott, D. et al. (1998). "PDDL — The Planning Domain Definition Language." Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control.

[3] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

[4] Berri AI. (2023). LiteLLM: Unified API for 100+ LLM providers. https://github.com/BerriAI/litellm

[5] Planning.Domains. *Online PDDL Solver Service*. https://solver.planning.domains

[6] Autonomous Software Agents Course, University of Trento. *Deliveroo.js — Course Scaffold*, A.A. 2025–2026. https://github.com/unitn-ASA/DeliverooJs